

VisionSpec QC

Automated PCB Defect Detection System

Full Project Report — Computer Vision Pipeline

Product	VisionSpec QC
Use Case	PCB Defect Detection
Backbone Model	MobileNetV2 (ImageNet pretrained)
Training Strategy	Two-Phase Transfer Learning
Deployment	TFLite + Flask Web Application
Production Requirement	>10 FPS, Defect Recall ≥ 0.90

1. Project Overview

VisionSpec QC is a four-week end-to-end computer vision pipeline designed to automate quality control on a PCB (Printed Circuit Board) manufacturing line. The system ingests raw PCB images, classifies them as PASS or DEFECT in real time, and produces interpretable Grad-CAM heatmaps showing exactly which regions of the board triggered the decision.

The pipeline was built with two hard production constraints in mind: a defect recall of at least 90% (missed defects are critical failures that could ship faulty hardware) and an inference throughput exceeding 10 FPS on a standard CPU (required to keep pace with a conveyor-line camera).

1.1 Production Constraints

Requirement	Target	Achieved
Defect Recall	≥ 0.90	✓ YES
Inference Speed (CPU)	> 10 FPS	✓ YES
Model Size (TFLite)	< 5 MB	✓ YES
Grad-CAM Overlay	Required	✓ YES
Web Application	Required	✓ YES

2. System Architecture

The pipeline is decomposed into four self-contained weekly modules, each producing artefacts consumed by the next stage. This separation makes the system easy to debug, retrain, or extend independently.

2.1 End-to-End Pipeline

Stage	Script	Outputs
Week 1	week1_data_preparation.py	Synthetic dataset (700+700 train, 150+150 val), augmentation config
Week 2	week2_model_training.py	Trained MobileNetV2 (.keras / .h5 / SavedModel), best threshold
Week 3	week3_grad_cam.py	Grad-CAM heatmaps, verification grid, deep-dive colourmap comparison
Week 4	week4_live_inference.py	TFLite quantised model (~3 MB), live demo with HUD, benchmark report
App	app.py	Flask REST API + production web dashboard at localhost:5000

2.2 Model Architecture

The model wraps MobileNetV2 (pretrained on ImageNet, alpha=1.0) with a lightweight classification head:

Input	224 × 224 × 3 (RGB, normalised [0, 1])
Backbone	MobileNetV2 — 3.4 M parameters, depth-wise separable convolutions
Feature Extraction	GlobalAveragePooling2D → 1280-d vector
Head Layer 1	BatchNormalization + Dense(256, ReLU) + Dropout(0.35)
Head Layer 2	Dense(128, ReLU) + Dropout(0.30)
Output	Dense(1, Sigmoid) → P(pass) ∈ [0, 1]
Total Parameters	~3.4 M backbone + ~360 K head

3. Model Selection: MobileNetV2 vs ResNet50

Two candidate backbones were evaluated before training began. MobileNetV2 was selected as the clear winner for production deployment.

Criterion	MobileNetV2	ResNet50	Winner
Parameters	3.4 M	25.6 M	MobileNetV2
Inference Speed (CPU)	> 30 FPS	~8 FPS	MobileNetV2
Model Size	~14 MB	~98 MB	MobileNetV2
Top-1 ImageNet Accuracy	71.8%	74.9%	ResNet50
Meets > 10 FPS req.	✓ YES	✗ NO	MobileNetV2
Memory Footprint	Low	High	MobileNetV2

Verdict: ResNet50's marginal 3% accuracy advantage does not justify its 7× parameter count or its failure to meet the > 10 FPS production requirement. MobileNetV2 meets all constraints with room to spare.

4. Week 1 — Synthetic Dataset Generation

4.1 Motivation

Real PCB defect datasets are scarce, expensive to label, and subject to confidentiality restrictions. A synthetic generator was built to produce visually realistic PCB images using PIL, with five distinct defect categories covering the most common manufacturing faults.

4.2 Dataset Configuration

Training set	700 PASS + 700 DEFECT = 1,400 images
Validation set	150 PASS + 150 DEFECT = 300 images
Image resolution	224 × 224 pixels (RGB)
PASS images	Clean grid, silver solder on all pads, $\sigma=3$ Gaussian noise
DEFECT images	1 mandatory defect + 30% chance of secondary; $\sigma=5$ noise
Dual-defect rate	30% of defect images contain two simultaneous faults

4.3 Defect Types

Defect Type	Visual Characteristic	Severity
Solder Bridge	Large gold blob (30–60 px) bridging adjacent pads, with overflow drips	HIGH
Missing Component	Empty copper pad, bright red border and X marker visible	HIGH
Trace Break	Dark gap in copper trace with scorched brown edges	MEDIUM
Burnt Spot	Concentric dark brown/black thermal scorch, 28–50 px radius, radiating streaks	HIGH
Cold Solder Joint	Dull grey rectangular joint with visible crack lines and grain texture	MEDIUM

4.4 Data Augmentation

Transform	Value	Rationale
rotation_range	$\pm 20^\circ$	PCB orientation variance on conveyor belt
zoom_range	15%	Camera distance variation between captures
brightness_range	[0.70, 1.30]	Factory lighting changes across shifts
horizontal_flip	True	PCBs can be loaded in either orientation

Transform	Value	Rationale
shear_range	4°	Lens distortion and perspective correction
width/height_shift	8%	Camera alignment drift
fill_mode	nearest	Avoids black border artefacts from transforms

5. Week 2 — Transfer Learning & Training

5.1 Two-Phase Training Strategy

Training is split into two phases to maximise convergence speed while minimising the risk of catastrophic forgetting of ImageNet features.

Phase	Backbone State	Learning Rate	Epochs	Purpose
Phase 1 (Feature Extraction)	All layers FROZEN	1×10^{-3}	Up to 25	Train custom head only
Phase 2 (Fine-tuning)	Top 20 layers UNFROZEN (BN frozen)	2×10^{-5}	Up to 20	End-to-end refinement

Phase 2 begins from the best Phase 1 checkpoint. All BatchNormalization layers inside MobileNetV2 remain frozen throughout fine-tuning to prevent corruption of the running statistics accumulated on ImageNet, which is critical for stability on a small synthetic dataset.

5.2 Training Configuration

Loss function	Binary Cross-Entropy
Optimiser	Adam
Class weights	{0: 1.2, 1: 1.0} (mild defect penalty — larger dataset carries recall load)
Early stopping	Patience 6 (Phase 1) / 8 (Phase 2), monitored on val_loss
LR scheduling	ReduceLROnPlateau — factor 0.3/0.5, patience 3/4, min_lr=1e-7
Model checkpoint	Best val_loss saved as .keras per phase
Batch size	32

5.3 Production Metrics at Evaluation

The evaluation procedure auto-scans thresholds from 0.25 to 0.80 and selects the value that maximises F1 score subject to recall ≥ 0.90 .

Defect Recall (Sensitivity)	$\geq 0.90 \leftarrow$ CRITICAL; missed defects ship faulty hardware
Defect Precision	Maximised within recall constraint; affects yield / false alarm rate
Specificity (Pass Recall)	Good boards wrongly rejected $\rightarrow$ wasted production capacity
F1 Score	Harmonic mean of precision and recall at optimal threshold

5.4 Saved Model Formats

Format	File	Use Case
.keras (native)	visionspec_qc_final.keras	Recommended; used by Week 3 Grad-CAM and Week 4 conversion
.h5 (legacy)	visionspec_qc_final.h5	Legacy compatibility with older TF tooling
SavedModel/	visionspec_qc_savedmodel/	TFServing and TFLite converter input
.tflite (Week 4)	visionspec_qc_quantized.tflite	Production inference — INT8 weights, ~3 MB

6. Week 3 — Grad-CAM Interpretability

6.1 Purpose

Grad-CAM (Gradient-weighted Class Activation Mapping) generates a spatial heatmap showing which regions of the board the model attended to when making its decision. This is essential for production use: quality control engineers must be able to verify that the model is looking at actual defects rather than background substrate or lighting artefacts.

6.2 Algorithm

For a single input image, Grad-CAM proceeds in four steps:

- Forward pass — record the feature maps at the final convolutional layer ('out_relu', the last inverted residual block output before GlobalAveragePooling2D).
- Gradient computation — compute the gradient of the defect class score with respect to each channel of that feature map using a GradientTape.
- Channel importance — pool the gradients spatially to obtain a scalar importance weight α_k for each channel k .
- Activation map — compute the ReLU-weighted sum of feature maps: $L_{cam} = \text{ReLU}(\sum \alpha_k \cdot A_k)$. Resize to input dimensions and apply a colour map.

The ReLU discards channels that suppress the defect score, keeping only those that increase it. The result is a focused heat map highlighting defect-contributing spatial regions.

6.3 Implementation Notes

MobileNetV2 uses a nested sub-model structure, which required a two-stage Grad-CAM model to ensure gradient flow:

- Stage 1: fresh input → conv_extractor → conv_out (watched explicitly by tape).
- Stage 2: conv_out → head layers → prediction (pred IS a function of conv_out).

Without this two-stage chain, `tape.gradient(pred, conv_out)` returns `None` because the prediction and the feature map originate from independent branches in the graph.

6.4 Heatmap Verification Criteria

Hot-pixel ratio (> 0.5 threshold)	Interpretation	Action
< 2%	Heatmap too sparse — model may not be attending meaningfully	Check layer name
2% – 40%	Focused heatmap — highlights specific component / pad regions	✓ Correct
> 40%	Heatmap too diffuse — may be attending to background substrate	Review layer selection

7. Week 4 — Inference Optimisation & Live Demo

7.1 Optimisation Pipeline

The trained Keras model is converted to TFLite with Dynamic Range Quantization to meet the > 10 FPS production requirement on a CPU-only deployment.

Input format	visionspec_qc_final.keras (float32, ~23 MB)
Conversion method	tf.lite.TFLiteConverter.from_keras_model + Optimize.DEFAULT
Quantization type	Dynamic Range Quantization — weights converted to INT8 at conversion
Output format	visionspec_qc_quantized.tflite (~3 MB, ~4× smaller)
Interpreter threads	max(4, os.cpu_count()) — uses all available physical cores
Startup warmup	3 dummy inferences to initialise XNNPACK JIT kernels before timing

7.2 Frame Pre-fetch Architecture

Synthetic frame generation using PIL costs ~20 ms per frame. Without pipelining, this overhead adds to the TFLite inference time (~80 ms), capping throughput at ~9.5 FPS. A double-buffered `FrameBuffer` class solves this:

- A background producer thread continuously generates frames and pushes them to a `Queue(maxsize=4)`.
- The main inference loop calls `queue.get()`, which returns instantly when the buffer is pre-filled.
- PIL generation is fully overlapped with TFLite inference → throughput increases to ~11.8 FPS.

7.3 Live Demo Controls

Key	Action
Q / Escape	Quit the demo
G	Toggle Grad-CAM overlay on/off
S	Save the current frame as a PNG file
Space	Pause / Resume the live feed

8. Web Application

8.1 Overview

A factory-floor quality control dashboard is served by a Flask backend (`app.py`). The UI supports drag-and-drop image upload, real-time PASS/DEFECT classification, Grad-CAM visualisation, session statistics, and an inspection history log. Three input modes are supported: single image upload, batch/folder processing, and live camera capture.

8.2 REST API Endpoints

Endpoint	Method	Description
/	GET	Renders the main dashboard (<code>templates/index.html</code>)
/api/inspect	POST	Upload image → TFLite inference → Grad-CAM → JSON result with base64 images
/api/history	GET	Last 50 inspection records (without image data for speed)
/api/stats	GET	Current session totals (total / pass / defect counts)
/api/reset	POST	Clear session statistics and inspection history

8.3 Response Payload

The `/api/inspect` endpoint returns a JSON object with the following fields:

Field	Type	Description
label	string	PASS or DEFECT
score	float	Raw sigmoid output $P(\text{pass}) \in [0, 1]$
defect_prob	float	$1 - \text{score}$ (human-readable defect probability)
confidence	float	Confidence in the predicted class
infer_ms	float	TFLite inference time in milliseconds
total_ms	float	Total processing time including Grad-CAM
thumb_b64	string	JPEG thumbnail of the original image (base64)
gradcam_b64	string	JPEG Grad-CAM overlay image (base64)
stats	object	Session totals: total, pass, defect, pass_rate, defect_rate

8.4 Deployment Notes

For production deployment, the Waitress WSGI server (Windows) or Gunicorn (Linux) should be used instead of the Flask development server. A single worker process (-w 1) is required because the TFLite interpreter and Grad-CAM model are not thread-safe across processes.

- Waitress (Windows/cross-platform): `waitress-serve --host=0.0.0.0 --port=5000 app:app`
- Gunicorn (Linux): `gunicorn -w 1 -b 0.0.0.0:5000 --timeout 120 app:app`

9. Project File Structure

Path	Description
app.py	Flask backend — inference API, session state, Grad-CAM route
week1_data_preparation.py	Synthetic PCB generator, augmentation pipeline
week2_model_training.py	MobileNetV2 two-phase training, evaluation, model export
week3_grad_cam.py	Grad-CAM engine, heatmap grid, deep-dive visualisation
week4_live_inference.py	TFLite conversion, live demo, benchmark runner
requirements.txt	Python dependencies
templates/index.html	Production web dashboard UI (HTML + CSS + JS)
models/visionspec_qc_final.keras	Final trained model (native Keras format)
models/visionspec_qc_final.h5	Final trained model (legacy HDF5 format)
models/visionspec_qc_quantized.tflite	Quantised inference model (~3 MB)
models/visionspec_qc_savedmodel/	TensorFlow SavedModel format for TFServing
models/best_threshold.txt	Auto-selected classification threshold from Week 2

Path	Description
results/week1_class_samples.png	Side-by-side PASS vs DEFECT sample visualisation
results/week1_augmented_batch.png	Augmented training batch grid
results/week2_learning_curves.png	Training/validation accuracy and loss curves
results/week2_confusion_matrix.png	Confusion matrix and threshold sensitivity curve
results/week3_gradcam_grid.png	Grad-CAM heatmap grid (4 pass + 4 defect)
results/week3_gradcam_detail.png	Single defect image with 4 colourmap options
test/T1–T5	Test images for manual inference verification
dataset/train/pass defect	Synthetic training images (700 per class)
dataset/val/pass defect	Synthetic validation images (150 per class)

10. Production Requirements Checklist

Requirement	Status
Real-time data augmentation via ImageDataGenerator	✓
Pre-trained MobileNetV2 backbone (ImageNet weights)	✓
Phase 1: Frozen backbone + custom classification head	✓
Phase 2: Fine-tuning with top-20 unfrozen layers	✓
BatchNorm layers frozen during fine-tuning (prevents stat corruption)	✓
Learning curves checked for overfitting	✓
EarlyStopping + ReduceLROnPlateau callbacks	✓
Auto-threshold selection maximising F1 at recall ≥ 0.90	✓
Grad-CAM on final Conv layer (out_relu)	✓
Heatmap verification (focused hot-pixel ratio 2–40%)	✓
TFLite Dynamic Range Quantization (INT8 weights)	✓
> 10 FPS production throughput (TFLite on CPU)	✓
OpenCV live demo with HUD overlay	✓
Grad-CAM background-threaded into live demo (non-blocking)	✓
Flask web application with drag-and-drop upload	✓
Batch / folder processing mode	✓
Live camera capture mode	✓
Session dashboard with live pass/defect statistics	✓

Requirement	Status
REST API for programmatic inspection access	✓
Model saved in .keras / .h5 / SavedModel / TFLite formats	✓

11. Disclaimer

This project is intended for research and educational purposes only. It is a demonstration of computer vision and transfer learning techniques applied to PCB defect detection and is NOT validated, certified, or approved for use in any real manufacturing, safety-critical, or production environment. Results may be inaccurate, incomplete, or unreliable when applied to real-world hardware. The authors accept no liability for any damages, losses, or defects arising from the use of this software. Always consult qualified engineers and conduct thorough testing before deploying any automated inspection system in a live setting.